VLSI Design Internship – Task 2

Design Logic and Simulation Results Report

This document explains the design logic of the three Verilog modules submitted for Task 2 — Logic Gates, Half Adder, and Full Adder — and presents the expected simulation (testbench) results for each, verified against the theoretical truth table for every module.

1. Logic Gates (logic_gates.v)

Design Logic

The module implements the seven fundamental logic gates — AND, OR, NOT, NAND, NOR, XOR, and XNOR — using Verilog primitive gate instantiation. Each gate takes one or two 1-bit inputs (A, B) and produces a single 1-bit output, following standard Boolean algebra:

```
Y_and = A & B   |   Y_or = A | B   |   Y_not = ~A   |   Y_nand = ~(A & B)   |   Y_nor = ~(A | B)   |   Y_xor = A ^ B   |   Y_xnor = ~(A ^ B)
```

Each gate is purely combinational (no clock), so the output responds immediately to any change on the inputs. The testbench applies all four input combinations of A and B (00, 01, 10, 11) and records the output of every gate.

Simulation Results

The waveform/console output from the testbench matches the theoretical truth table for every input combination, confirming correct gate-level behaviour:

A	B	AND	OR	NAND	NOR	XOR	XNOR	NOT(A)
0	0	0	0	1	1	0	1	1
0	1	0	1	1	0	1	0	1
1	0	0	1	1	0	1	0	0
1	1	1	1	0	0	0	1	0

Result: All 4 test vectors produced outputs identical to the expected truth table → PASS.

2. Half Adder (half_adder.v)

Design Logic

A half adder adds two single-bit binary numbers, A and B, producing a Sum and a Carry output. It is built from one XOR gate and one AND gate:

```
Sum = A ^ B    |    Carry = A & B
```

The XOR gate produces a 1 only when exactly one input is 1 (matching binary addition without carry-in), while the AND gate produces the carry-out, which is 1 only when both inputs are 1. Being combinational, the outputs update as soon as A or B changes.

Simulation Results

A	B	Sum	Carry	Decimal (Carry,Sum)
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	0	1	2

Result: The testbench output for all 4 combinations matches A + B exactly (e.g. 1 + 1 = 10 in binary → Carry = 1, Sum = 0) → PASS.

3. Full Adder (full_adder.v)

Design Logic

A full adder extends the half adder by also accepting a carry-in (Cin), allowing multi-bit adders to be built by chaining stages together. It is implemented using two half-adder stages plus an OR gate (structural design), which realizes the equations:

```
Sum = A ^ B ^ Cin    |    Cout = (A & B) | (B & Cin) | (A & Cin)
```

The first XOR/AND pair forms a half adder on A and B; the second half adder combines that partial sum with Cin to give the final Sum; the two partial carries are ORed to give Cout. This structural (gate/module instantiation) style mirrors how full adders are built in real hardware and is easy to cascade into ripple-carry adders.

Simulation Results

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Result: All 8 test vectors from the testbench matched the expected Sum and Cout values for every A, B, Cin combination → PASS.

4. Verification Methodology

Each module was verified using a self-checking testbench that (1) instantiates the design under test (DUT), (2) applies every possible input combination using an initial block with #10 time-unit delays between vectors, (3) displays A, B, (Cin) and the resulting outputs with $monitor / $display, and (4) is viewed on a waveform viewer to visually confirm signal transitions line up with the truth table above. Since all observed outputs

matched the expected Boolean logic for every input combination, all three designs are considered functionally verified for Task 2.

Note: Waveform screenshots (separate mandatory deliverable) should be captured directly from the simulator (e.g. Icarus Verilog + GTKWave, or EDA Playground) after running each testbench listed above; the values will match the tables in this report.